Aurora Traversini

Tesina di
Signal Processing and Optimization for Big Data
Prof. Paolo Banelli

Matrix Completion per Recommender Systems: Analisi dell'algoritmo SGD su Dataset MovieLens 1M

Perugia, Anno Accademico 2025/2026
Università degli Studi di Perugia
Corso di laurea magistrale in Ingegneria Informatica e Robotica
Curriculum Data Science
Dipartimento di Ingegneria



A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

0. Indice

1 Introduzione **3**

2 Concetti teorici **4**

2.1 Formulazione del problema 4

2.2 Ottimizzazione tramite SGD 5

2.3 Strategie di Inizializzazione 6

2.3.1 Random Initialization 7

2.3.2 Spectral Initialization (SVD-based) 7

3 Implementazione **8**

4 Verifica Sperimentale e Analisi **10**

4.1 Dataset: MovieLens 1M 10

4.2 Fase 1: Confronto Inizializzazioni (Random vs Spectral) 10

4.3 Fase 2: Grid Search Multidimensionale (α e β) 11

4.4 Fase 3: Rank Sensitivity 13

4.5 Fase 4: Diagnostica Visiva della Matrix Completion 15

5 Conclusioni **17**

6 Bibliografia **19**

1. Introduzione

Lo scopo di questa tesina è l'implementazione dell'algoritmo *Stochastic Gradient Descent (SGD)* per la *Low-Rank Matrix Factorization*, al fine di stimare i valori mancanti di una matrice.

In particolare, la *Matrix Completion*, centrale nell'era dei Big Data, viene qui applicata al contesto dei sistemi di raccomandazione (Recommender Systems) per prevedere il voto che un utente assegnerebbe a un film che non ha ancora visto, e che dunque non ha valutato, sfruttando una matrice Utenti-Film che è estremamente sparsa.

Il focus principale del lavoro è mostrare come strategie di inizializzazione intelligenti, basate sulla decomposizione ai valori singolari (*Spectral Initialization*), possano migliorare drasticamente la convergenza e la stabilità dell'algoritmo rispetto a un'inizializzazione casuale. Inoltre, viene analizzato l'impatto degli iperparametri (rango, learning rate e regolarizzazione) tramite un approccio di grid search multidimensionale, utilizzando dati reali provenienti dal dataset *MovieLens 1M*.

2. Concetti teorici

2.1 Formulazione del problema

Sia $R \in \mathbb{R}^{m \times n}$ la matrice dei rating, dove m rappresenta il numero di utenti e n il numero di film. Nel contesto dei sistemi di raccomandazione reali, questa matrice è estremamente **sparsa**: conosciamo i valori solo per un sottoinsieme limitato di indici, che definiamo come l'insieme di campionamento Ω:

$$\Omega \subseteq \{1, \dots, m\} \times \{1, \dots, n\}$$

Possiamo quindi definire l'insieme delle entry a noi note come $R_\Omega = \{R_{ij} : (i, j) \in \Omega\}$. L'obiettivo del problema di *Matrix Completion* è predire o ricostruire le entry mancanti della matrice R sfruttando esclusivamente la conoscenza del sottoinsieme di entry note R_Ω.

Per formalizzare matematicamente l'osservazione parziale, introduciamo l'operatore di proiezione sull'insieme Ω, indicato con $\mathcal{P}_\Omega$. Il suo effetto sulla matrice dei rating R è definito in questo modo:

$$[\mathcal{P}_\Omega(R)]_{i,j} = \begin{cases} R_{ij}, & \text{se } (i, j) \in \Omega \\ 0, & \text{altrimenti} \end{cases}$$

Il problema della ricostruzione ha senso ed è risolvibile solo se i dati presentano una struttura sottostante. Si ipotizza quindi che la matrice completa R non sia a rango pieno, ma abbia un **rango basso** $r \ll \min(m, n)$.

Questa assunzione implica l'esistenza di un numero limitato di *fattori latenti* che modellano i dati che stiamo osservando e che, nel nostro caso, governano le preferenze degli utenti e le caratteristiche intrinseche dei film. Tanto più il rango è basso, minori saranno i gradi di libertà del sistema; questo significa che le preferenze degli utenti non sono del tutto indipendenti tra loro, ma seguono dei pattern comuni che permettono al modello di dedurre fedelmente l'intera matrice a partire da un numero ridotto di osservazioni.

Lo scopo è quindi trovare due matrici di fattori latenti di dimensioni ridotte, $U \in \mathbb{R}^{m \times r}$ (matrice utenti) e $V \in \mathbb{R}^{n \times r}$ (matrice film), tali che il loro prodotto approssimi la matrice originale $R \approx UV^T$. Nel nostro modello di ottimizzazione, ogni singolo voto stimato $\hat{r}_{ij}$ è dato dal prodotto scalare tra il vettore dei fattori latenti dell'utente i e quello del film j:

$$\hat{r}_{ij} = u_i^T v_j = \sum_{k=1}^{r} u_{ik} v_{jk}$$

2.2 Ottimizzazione tramite SGD

Il problema di fattorizzazione viene tipicamente regolarizzato per evitare l'overfitting sui dati noti. L'obiettivo è minimizzare la funzione di costo C, definita come la norma di Frobenius dell'errore sui rating osservati, sommata a un termine di regolarizzazione.

Sfruttando l'operatore di proiezione $\mathcal{P}_\Omega$ precedentemente definito, il problema di minimizzazione matriciale è espresso formalmente come:

$$\arg\min_{U,V} \left\{ \|\mathcal{P}_\Omega(UV^T - R)\|_F^2 + \beta(\|U\|_F^2 + \|V\|_F^2) \right\}$$

Questa formulazione globale equivale alla minimizzazione della somma dei quadrati calcolata sui singoli elementi appartenenti al set di campionamento Ω:

$$C(U,V) = \sum_{(i,j)\in\Omega} (\underline{u}_i^T \underline{v}_j - r_{ij})^2 + \beta(\|\underline{u}_i\|^2 + \|\underline{v}_j\|^2)$$

Per procedere con il *Gradient Descent*, calcoliamo le derivate parziali della funzione di costo rispetto ai vettori dei fattori latenti. Mostrando i passaggi della regola della catena

come indicato nella teoria, otteniamo:

$$\frac{\partial C}{\partial \underline{u}_i} = 2(\underline{u}_i^T \underline{v}_j - r_{ij}) \frac{\partial (\underline{u}_i^T \underline{v}_j)}{\partial \underline{u}_i} + 2\beta \underline{u}_i = 2(\underline{u}_i^T \underline{v}_j - r_{ij}) \underline{v}_j + 2\beta \underline{u}_i \tag{2.1}$$

$$\frac{\partial C}{\partial \underline{v}_j} = 2(\underline{u}_i^T \underline{v}_j - r_{ij}) \frac{\partial (\underline{u}_i^T \underline{v}_j)}{\partial \underline{v}_j} + 2\beta \underline{v}_j = 2(\underline{u}_i^T \underline{v}_j - r_{ij}) \underline{u}_i + 2\beta \underline{v}_j \tag{2.2}$$

Utilizzando lo *Stochastic Gradient Descent (SGD)*, le regole di aggiornamento al passo iterativo k diventano quindi:

$$\underline{u}_i^{(k)} \leftarrow \underline{u}_i^{(k-1)} - 2\alpha \left[(\underline{u}_i^{T(k-1)} \underline{v}_j^{(k-1)} - r_{ij}) \underline{v}_j^{(k-1)} + \beta \underline{u}_i^{(k-1)} \right] \tag{2.3}$$

$$\underline{v}_j^{(k)} \leftarrow \underline{v}_j^{(k-1)} - 2\alpha \left[(\underline{u}_i^{T(k-1)} \underline{v}_j^{(k-1)} - r_{ij}) \underline{u}_i^{(k-1)} + \beta \underline{v}_j^{(k-1)} \right] \tag{2.4}$$

Dove α rappresenta il *learning rate* (passo di apprendimento) e β è il parametro di regolarizzazione. In questa formulazione esatta il gradiente spinge i parametri nella direzione opposta alla derivata (segno meno) riducendo progressivamente la distanza tra il prodotto scalare latente e il rating reale.

2.3 Strategie di Inizializzazione

Il problema di ottimizzazione per la fattorizzazione matriciale non ammette una soluzione globale in forma chiusa poiché la funzione di costo, a causa del prodotto tra le matrici incognite U e V, è congiuntamente non convessa.

Come discusso nella teoria di riferimento del corso, il problema risulta tuttavia *separatamente convesso*: se si fissa una delle due matrici (ad esempio V), l'ottimizzazione rispetto all'altra (U) diventa un problema convesso assimilabile a una *Ridge Regression*, risolvibile in forma chiusa. Questo è il principio alla base di approcci alternativi all'ottimizzazione, come l'*Alternating Least Squares* (ALS), che però non verrà trattato in questa tesina.

Tuttavia, procedendo con l'algoritmo SGD per l'ottimizzazione congiunta e simultanea dei fattori, la natura congiuntamente non convessa della funzione di costo rende la discesa del gradiente estremamente sensibile al punto di partenza, esponendo il modello al forte rischio di convergere verso minimi locali subottimali. Per questo

motivo, la scelta della strategia di inizializzazione assume un ruolo critico per il successo della Matrix Completion.

2.3.1 Random Initialization

È l'approccio standard: le matrici U e V vengono riempite con valori casuali estratti da una distribuzione normale con media nulla e varianza ridotta (impostata tipicamente, come nell'implementazione proposta, inversamente proporzionale al rango, ossia con deviazione standard pari a $1/r$). Questo *scaling* iniziale è un passaggio matematico fondamentale per evitare l'esplosione dei gradienti nelle prime iterazioni: inizializzare le matrici con valori casuali non controllati porterebbe a stime iniziali dei rating (date dal prodotto scalare $u_i^T v_j$) sproporzionate, causando forte instabilità o divergenza immediata dell'algoritmo. Pur essendo di semplice implementazione, questo metodo risulta spesso instabile e porta a convergenze lente, specialmente su dataset ad alta sparsità dove il paesaggio di ottimizzazione è complesso.

2.3.2 Spectral Initialization (SVD-based)

Questa tecnica sfrutta l'algebra lineare per partire da una stima che sia già vicina alla soluzione ottima globale. Si calcola la decomposizione ai valori singolari troncata (*Truncated SVD*) della matrice sparsa osservata R_Ω:

$$R_\Omega \approx U_r \Sigma_r V_r^T$$

Le matrici dei fattori latenti vengono inizializzate bilanciando simmetricamente l'energia dei valori singolari, fornendo a SGD un punto di partenza informato:

$$U^{(0)} = U_r \Sigma_r^{1/2}, \quad V^{(0)} = V_r \Sigma_r^{1/2}$$

3. Implementazione

Il progetto è stato sviluppato in ambiente Python, facendo ampio uso delle librerie scientifiche `NumPy` e `SciPy`. In particolare, la gestione della matrice dei rating, caratterizzata da un'elevatissima sparsità (superiore al 95%), ha richiesto l'utilizzo del modulo `scipy.sparse` per ottimizzare l'occupazione di memoria e l'efficienza computazionale. Per l'inizializzazione spettrale, ci si è avvalsi della funzione `svds` per il calcolo rapido della decomposizione troncata ai valori singolari.

Di seguito viene riportato il frammento principale del codice (Code Snippet 3.1), che implementa il ciclo di addestramento dell'algoritmo SGD.

```
def run_sgd_optimized(n_u, n_m, rank, init_method,
    R_sparse, ...):
     # 1. Inizializzazione Spettrale o Randomica
     if init_method == 'spectral':
         U, V = spectral_initialization(R_sparse, rank)
     else:
         # La varianza e' scalata in base al rango (1/r)
         U = np.random.normal(scale=1./rank, size=(n_u,
            rank))
         V = np.random.normal(scale=1./rank, size=(n_m,
            rank))

     # 2. Ciclo di Training Stocastico
     for step in range(steps):
```

```
        # Permutazione casuale per la stocasticita del
           gradiente
        shuffled_indices =
           np.random.permutation(len(t_ratings))

        for idx in shuffled_indices:
            u, i, r = t_users[idx], t_movies[idx],
               t_ratings[idx]

            # Calcolo errore e aggiornamento gradienti
            pred = np.dot(U[u], V[i])
            err = r - pred

            U[u] += alpha * (err * V[i] - beta * U[u])
            V[i] += alpha * (err * U[u] - beta * V[i])

    return history, U, V
```

Code Snippet 3.1: Implementazione del core SGD in Python

Come si evince dal codice, l'algoritmo prevede due fasi principali. La prima consiste nell'inizializzazione delle matrici U e V in base al metodo scelto, gestendo esplicitamente lo *scaling* della varianza nel caso randomico.

La seconda fase è il ciclo di ottimizzazione vero e proprio: ad ogni iterazione (epoca), gli indici delle valutazioni note vengono permutati casualmente tramite la funzione `np.random.permutation`. Questo passaggio è cruciale per rispettare la natura "stocastica" della discesa del gradiente, in quanto garantisce che i campioni vengano presentati al modello in ordine sempre diverso, riducendo il rischio di instabilità e migliorando la traiettoria di convergenza.

Infine, le regole di aggiornamento traducono fedelmente le equazioni matematiche del gradiente ricavate nel capitolo teorico. Si noti come, a livello implementativo, l'errore sia calcolato come differenza tra il valore reale e la predizione $(r - \hat{r})$ e il fattore moltiplicativo 2 della derivata teorica sia stato direttamente assorbito all'interno dell'iperparametro `alpha` (il *learning rate*), una pratica standard nell'ottimizzazione del Machine Learning.

4. Verifica Sperimentale e Analisi

4.1 Dataset: MovieLens 1M

L'analisi sperimentale è stata condotta sul dataset **MovieLens 1M**, uno standard di riferimento per i sistemi di raccomandazione, contenente circa 1 milione di valutazioni.

- **Utenti:** 6.040
- **Film:** 3.706
- **Sparsità:** 95.53% (la matrice dei rating è dunque quasi completamente vuota).

4.2 Fase 1: Confronto Inizializzazioni (Random vs Spectral)

Per giustificare la scelta dell'approccio adottato, è stato effettuato un confronto preliminare mantenendo fissi gli iperparametri ($r = 50, \alpha = 0.005, \beta = 0.02$). I risultati hanno dimostrato una netta superiorità dell'inizializzazione spettrale: grazie a una stima iniziale informata basata su SVD troncata, l'algoritmo mostra una convergenza più rapida, un RMSE finale più basso e un comportamento più stabile rispetto alla controparte randomica, che tende a presentare lunghe fasi di stagnazione.

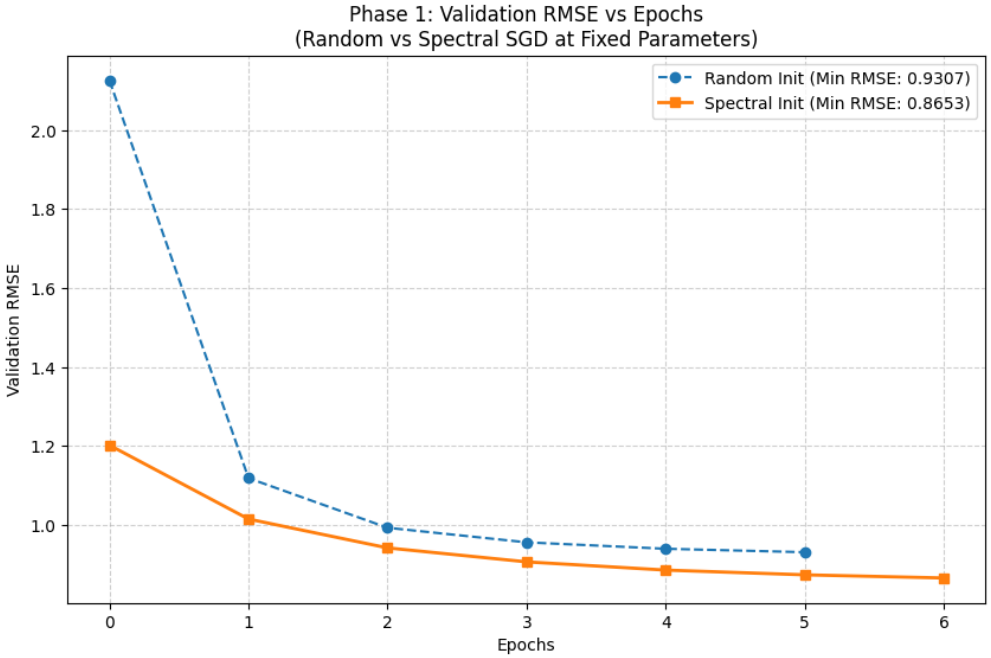


Figura 4.1: Effetto dell'inizializzazione Random rispetto a quella Spettrale sulla curva di apprendimento.

4.3 Fase 2: Grid Search Multidimensionale (α e β)

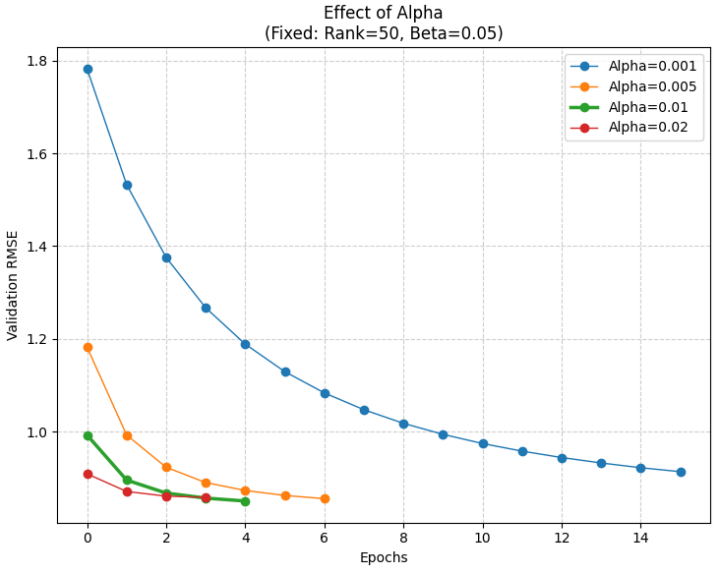
Poiché i parametri di learning rate α e di regolarizzazione β influenzano congiuntamente sia la stabilità che la velocità di convergenza, è stata condotta una **Grid Search bidimensionale** su questi due iperparametri, mantenendo fisso un rango iniziale ragionevole ($r = 50$). Nello specifico, lo spazio di ricerca è stato definito testando i valori $\alpha \in \{0.001, 0.005, 0.01, 0.02\}$ e $\beta \in \{0.01, 0.02, 0.05, 0.1\}$. Questa scelta ha permesso di evitare una ricerca simultanea a tre dimensioni (includendo anche il rango) che, date le dimensioni del dataset (circa un milione di interazioni), avrebbe comportato un costo computazionale proibitivo. Separare la calibrazione della discesa del gradiente dalla ricerca sulla complessità del modello ha reso l'analisi più mirata ed efficiente.

Tra le combinazioni esplorate, la coppia che ha mostrato il miglior compromesso tra rapidità di convergenza e stabilità numerica è risultata essere $\boldsymbol{\alpha = 0.01}$ e $\boldsymbol{\beta = 0.05}$. Fissando alternativamente l'uno e l'altro valore ottimale, le curve di sensibilità hanno confermato visivamente questa scelta. Come si evince chiaramente dalla Figura 4.2a, un

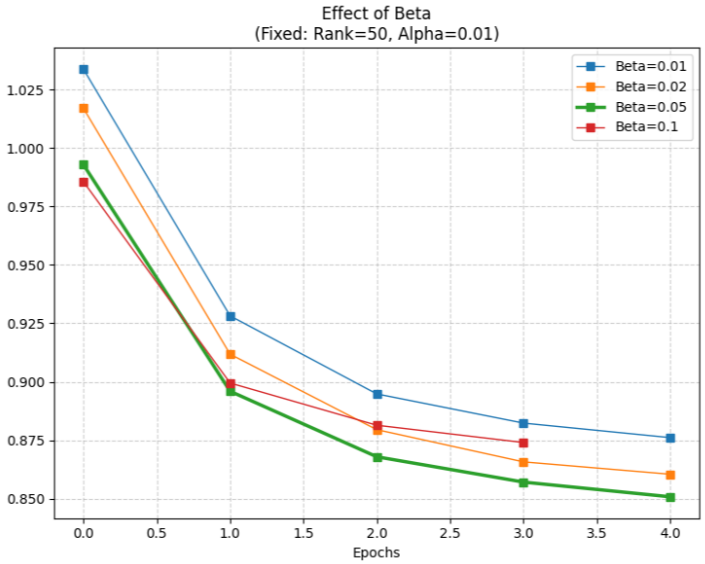
passo di apprendimento troppo conservativo (es. $\alpha = 0.001$) porta a una discesa estremamente lenta, mentre un valore eccessivo ($\alpha = 0.02$) fa scattare precocemente le condizioni di arresto (*early stopping*): il modello converge prematuramente a un errore subottimale poiché l'eccessiva ampiezza del passo di gradiente genera instabilità, impedendo di affinare ulteriormente la stima.

Analogamente, la Figura 4.2b dimostra come una regolarizzazione troppo blanda ($\beta = 0.01$) rallenti l'abbattimento dell'errore, mentre una penalizzazione troppo severa ($\beta = 0.1$) blocchi eccessivamente l'aggiornamento dei pesi causando un chiaro fenomeno di *underfitting* (la curva si appiattisce su valori di RMSE più alti rispetto all'ottimo).

Questa duplice analisi visiva conferma l'ottimalità della coppia scelta per garantire la discesa dell'errore più profonda e stabile.



(a) Effetto di α con β fissato.



(b) Effetto di β con α fissato.

Figura 4.2: Confronto tra le curve di sensibilità dei parametri α e β.

4.4 Fase 3: Rank Sensitivity

Individuata la coppia di parametri ottimale per la discesa del gradiente, si è analizzato l'impatto della complessità del modello variando il numero di fattori latenti

$r \in \{10, 20, 50, 100\}$.

È doveroso fare una premessa metodologica: avendo ottimizzato gli iperparametri α e β mantenendo fisso il rango a 50, è lecito attendersi che il modello performi al meglio proprio in questa configurazione. Teoricamente, aumentare il rango fornisce al modello una maggiore "memoria" per rappresentare i dati ma, su dataset caratterizzati da alta sparsità, un numero eccessivo di parametri liberi spinge rapidamente il modello verso l'*overfitting*, evidenziando il classico trade-off bias-varianza.

Proprio per garantire un confronto più equo e contrastare questo fenomeno, è stata implementata un'**euristica di regolarizzazione**: per ranghi critici ($r \geq 100$), il valore di β è stato raddoppiato rispetto all'ottimo precedentemente individuato (da 0.05 a 0.1). Questa scelta mira a compensare l'aumento dei gradi di libertà, fornendo al modello una penalizzazione più severa per mitigare la memorizzazione del rumore.

Come si evince dalla Figura 4.3, i modelli con ranghi inferiori ($r = 10$ e $r = 20$) convergono a valori di errore più alti, manifestando una limitata capacità espressiva (*underfitting*). D'altro canto, nonostante la contromisura della regolarizzazione euristica per compensare il bias dei parametri iniziali, il modello ha mostrato prestazioni peggiori a $r = 100$ rispetto a $r = 50$. Questo risultato dimostra empiricamente che i dati del dataset MovieLens 1M non contengono una densità informativa sufficiente per giustificare un numero così elevato di fattori latenti. Il rango ottimale globale confermato per questa architettura è pertanto $\boldsymbol{r = 50}$, che garantisce il valore di RMSE in validazione più basso.

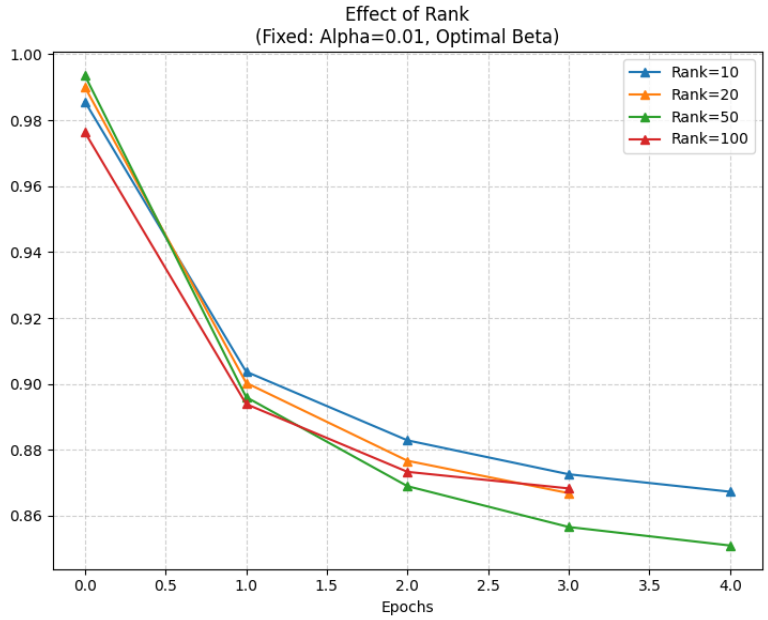


Figura 4.3: Learning Curves al variare del rango e curva del Trade-off Bias-Varianza.

4.5 Fase 4: Diagnostica Visiva della Matrix Completion

Infine, una volta addestrato il modello finale con la tripletta ottima ($r = 50, \alpha = 0.01, \beta = 0.05$), si è analizzato qualitativamente il comportamento della fattorizzazione isolando un sottoinsieme rappresentativo della matrice (nello specifico, una porzione dimensionata a 50×50, comprendente i primi 50 utenti e 50 film). Questa estrazione locale è strettamente necessaria ai fini diagnostici: visualizzare l'intera matrice originale renderebbe impossibile distinguere le singole interazioni a causa dell'estrema densità visiva dei punti.

È stata creata una visualizzazione a 4 pannelli per illustrare il processo di completamento:

1. **Original Data:** Mostra la forte sparsità di partenza e la disposizione dei voti noti.
2. **Training Fit:** Verifica che il modello riesca a replicare fedelmente i valori noti senza collassare in un fenomeno di *underfitting*.
3. **Prediction (Inference):** Isola i valori dedotti matematicamente dal modello per riempire esclusivamente i gap informativi (i valori nulli originali).

4. **Completed Matrix:** Il risultato finale denso, da cui il Recommender System può estrarre le raccomandazioni (es. ordinando la riga di un utente per proporre i rating predetti più alti sui film non ancora visti).

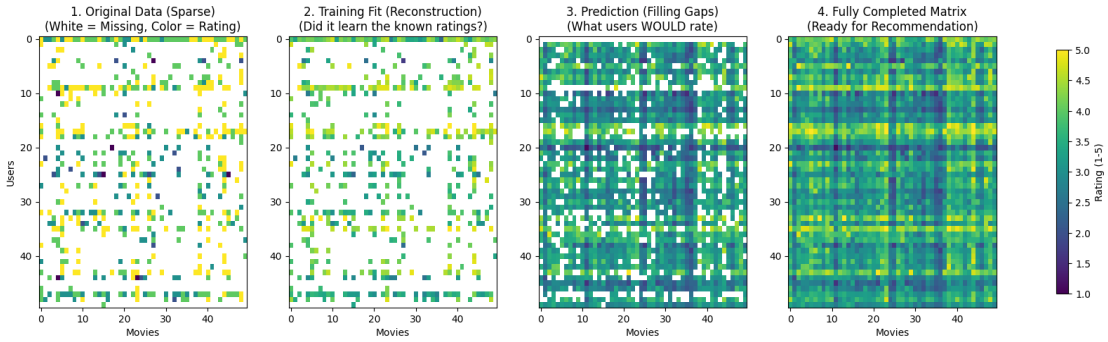


Figura 4.4: Visualizzazione a 4 pannelli del processo di Matrix Completion.

5. Conclusioni

In questo lavoro si è affrontato il problema della *Matrix Completion* applicata ai sistemi di raccomandazione, implementando un algoritmo di fattorizzazione basato su *Stochastic Gradient Descent* (SGD). L'analisi condotta sul dataset MovieLens 1M ha permesso di trarre conclusioni rilevanti sia dal punto di vista metodologico che prestazionale:

- **Inizializzazione con SVD:** L'utilizzo della *Spectral Initialization* si è rivelato un fattore determinante. A differenza dell'approccio randomico, la partenza basata su SVD permette di superare le criticità legate alla non convessità congiunta della funzione di costo, garantendo una convergenza molto più rapida verso minimi di errore, evitando fasi di stallo numerico.
- **Interazione tra iperparametri:** La fase di *Grid Search* ha confermato che il *learning rate* e la regolarizzazione non sono parametri indipendenti, dunque un'ottimizzazione efficace richiede un bilanciamento fine: un passo troppo ampio genera instabilità che nemmeno una regolarizzazione elevata riesce a compensare, mentre una penalizzazione eccessiva conduce inevitabilmente all'underfitting delle matrici latenti.
- **Capacità del modello e trade-off Bias-Varianza:** L'analisi della *Rank Sensitivity* ha dimostrato che la complessità del modello deve essere commisurata alla densità informativa del dataset. Nonostante l'adozione di strategie di regolarizzazione euristica per i modelli a rango elevato, il valore $r = 50$ è risultato l'ottimo globale. Superare tale valore introduce una varianza eccessiva che il sistema non riesce a gestire, portando a un decremento della capacità di generalizzazione (overfitting).

La diagnostica visiva finale, eseguita su un campione rappresentativo, ha validato qualitativamente l'efficacia del modello nel colmare le lacune informative della matrice sparsa. In conclusione, il sistema sviluppato non solo dimostra solidità matematica nella stima dei rating mancanti, ma fornisce un'architettura robusta e scalabile, pronta per essere integrata in motori di raccomandazione capaci di operare su scenari reali di *Big Data*.

6. Bibliografia

[1] Hastie, T., Tibshirani, R., & Wainwright, M. (2015). *Statistical Learning with Sparsity*. CRC Press.

[2] Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*.